```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// ========================================
// ⚙️ CONFIGURATION
// ========================================
const char* ssid = "iPhone-F";         // <-- CHANGE THIS
const char* password = "zeina200506";  // <-- CHANGE THIS

const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
// Change const char* to String
String node = "A";
String topic = "guc/ward1/esp_node_" + node + "/vitals";
String command_topic = "guc/ward1/esp_node_" + node + "/alarm";


// Define Button Pins
const int BTN_SAFE = 13;      // Pin for Safe
const int BTN_SMALL = 12;     // Pin for Small Deviation
const int BTN_LARGE = 14;     // Pin for Large Deviation
const int BTN_CRITICAL = 27;  // Pin for Critical
const int LED_RED = 25;       // New: Red LED for Critical
const int LED_GREEN = 26;     // New: Green LED for Safe



char lastJson[256] = "";
const int LED_BUILTIN = 2;
// MQTT Client Setup
WiFiClient espClient;
PubSubClient client(espClient);

// ========================================
// 🛠️ SETUP
// ========================================
void setup() {
  Serial.begin(115200);

  // Set button pins as inputs with internal pull-up resistors
  // (Connect buttons between the pin and GND)
  pinMode(BTN_SAFE, INPUT_PULLUP);
  pinMode(BTN_SMALL, INPUT_PULLUP);
  pinMode(BTN_LARGE, INPUT_PULLUP);
  pinMode(BTN_CRITICAL, INPUT_PULLUP);
  pinMode(LED_RED,OUTPUT);
  pinMode(LED_GREEN,OUTPUT);
  digitalWrite(LED_GREEN,HIGH);
  digitalWrite(LED_RED,LOW);

  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
  client.setCallback(mqttCallback);
```

```
  // Seed the random number generator
  randomSeed(analogRead(0));
}

void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    String clientId = "ESP32Client-";
    clientId += String(random(0xffff), HEX);

    if (client.connect(clientId.c_str())) {
      Serial.println("connected");

      // --- Subscribe to the command topic for this node ---
      String commandTopic = String("guc/ward1/esp_node_A/alarm");
      client.subscribe(commandTopic.c_str());
      Serial.print("📡 Subscribed to: ");
      Serial.println(commandTopic);

      // Optional: send a default safe payload
      generateAndSend("safe");
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void mqttCallback(char* topic, byte* payload, unsigned int length) {
  // Convert payload to a string
  payload[length] = '\0';                       // Null terminate
  String message = String((char*)payload);

  Serial.print("📨 Command received on topic: ");
```

```
  Serial.print(topic);
  Serial.print(" -> ");
  Serial.println(message);

  // Check if it's the alarm command topic for this node
  if (String(topic) == command_topic) {
    if (message == "CRITICAL") {
      // Turn on an LED or buzzer (example: built-in LED)
      digitalWrite(LED_BUILTIN, HIGH);    // Or any other output pin
      Serial.println("🚨 CRITICAL ALARM ACTIVATED");
      digitalWrite(LED_RED,HIGH);
      digitalWrite(LED_GREEN,LOW);
    }
    else if (message == "SAFE") {
      digitalWrite(LED_BUILTIN, LOW);
      Serial.println("✅ Alarm cleared");
      digitalWrite(LED_RED,LOW);
      digitalWrite(LED_GREEN,HIGH);
    }
  }
}

// ==========================================
// 🧱 HELPER: RANDOM FLOAT GENERATOR
// ==========================================
float randomFloat(float min, float max) {
  return ((float)random(0, 10000) / 10000.0) * (max - min) + min;
}

// ==========================================
// 🚀 PAYLOAD GENERATOR & PUBLISHER
// ==========================================
void generateAndSend(String severity) {
  float bpm, spo2, temp, acc_x, acc_y, acc_z;

  // Default Accelerometer (Resting in bed)
  acc_x = randomFloat(-0.1, 0.1);
  acc_y = randomFloat(-0.1, 0.1);
  acc_z = randomFloat(0.9, 1.1);

  if (severity == "safe") {
    bpm = randomFloat(70.0, 85.0);
    spo2 = randomFloat(96.0, 100.0);
    temp = randomFloat(36.5, 37.2);
  }
  else if (severity == "small") {
    bpm = (random(0, 2) == 0) ? randomFloat(55.0, 65.0) :
randomFloat(90.0, 105.0);
    spo2 = randomFloat(93.0, 95.0);
    temp = randomFloat(37.5, 38.2);
  }
  else if (severity == "large") {
    bpm = (random(0, 2) == 0) ? randomFloat(45.0, 54.0) :
randomFloat(110.0, 125.0);
```

```
    spo2 = randomFloat(89.0, 92.0);
    temp = randomFloat(38.5, 39.5);
  }
  else if (severity == "critical") {
    bpm = (random(0, 2) == 0) ? randomFloat(30.0, 40.0) :
randomFloat(130.0, 160.0);
    spo2 = randomFloat(80.0, 87.0);
    temp = randomFloat(40.0, 41.5);
    // Simulate a fall
    acc_x = randomFloat(1.5, 3.0);
    acc_y = randomFloat(1.5, 3.0);
    acc_z = randomFloat(0.0, 0.5);
  }

  // Create JSON Payload
  StaticJsonDocument<256> doc;
  doc["bpm"] = round(bpm * 10.0) / 10.0;
  doc["spo2"] = round(spo2 * 10.0) / 10.0;
  doc["temp_c"] = round(temp * 10.0) / 10.0;

  JsonObject accel = doc.createNestedObject("accel");
  accel["x"] = round(acc_x * 100.0) / 100.0;
  accel["y"] = round(acc_y * 100.0) / 100.0;
  accel["z"] = round(acc_z * 100.0) / 100.0;

  // Serialize to String
  char jsonBuffer[256];
  serializeJson(doc, jsonBuffer);
  strcpy(lastJson, jsonBuffer);
  // Publish to MQTT
client.publish(topic.c_str(), jsonBuffer);

  Serial.print("📡 Sent ");
  Serial.print(severity);
  Serial.println(" payload:");
  Serial.println(jsonBuffer);
}

// ==========================================
// 🔁 MAIN LOOP
// ==========================================
void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

    // ---------- Periodic Ping (every 5 seconds) ----------
  static unsigned long lastPingTime = 0;
  if (millis() - lastPingTime >= 5000) {
    if (client.connected() && strlen(lastJson) > 0) {
      client.publish(topic.c_str(), lastJson);
      Serial.println("🔁 Ping: resent last payload");
    }
```

```
    lastPingTime = millis();
  }

  // Read buttons (LOW means pressed because we use INPUT_PULLUP)
  if (digitalRead(BTN_SAFE) == HIGH) {
    Serial.print("✅ Sent 🟢 SAFE ");
    generateAndSend("safe");
    lastPingTime = millis();
    delay(500); // Simple debounce
  }
  else if (digitalRead(BTN_SMALL) == HIGH) {
    Serial.print("✅ Sent 🟡 SMALL DEVIATION ");
    generateAndSend("small");
    lastPingTime = millis();
    delay(500);
  }
  else if (digitalRead(BTN_LARGE) == HIGH) {
    Serial.print("✅ Sent 🟠 LARGE DEVIATION ");
    generateAndSend("large");
    lastPingTime = millis();
    delay(500);
  }
  else if (digitalRead(BTN_CRITICAL) == HIGH) {
    Serial.print("✅ Sent 🔴 CRITICAL ");
    generateAndSend("critical");
    lastPingTime = millis();
    delay(500);
  }
}
```